Project Arc — Current User Workflow

Comprehensive Architectural Startup & Data Flow Audit

1. Application Startup & Initialization Sequence

Initialization Entry Point: lib/main.dart → main()
Upon launch, the application initializes Flutter bindings, timezone data, Firebase Core (Firebase.initializeApp()), local Hive storage (HiveService().init()), and local notification services. It mounts a Riverpod ProviderScope wrapping SyncInitializer as the root widget surrounding ProjectArcApp.

2. Actual vs. Intended Workflow Diagram

INTENDED WORKFLOW	ACTUAL IMPLEMENTATION (AUDITED)
APP LAUNCH ↓ Firebase Auth Check (currentUser) ■■■ **NO AUTH** → Display **Login Screen** (/login) ■■■ **AUTHENTICATED** ↓ Open User-Scoped Hive Boxes ↓ Load Local Data → Render Main App ↓ Background Firestore Sync	APP LAUNCH ↓ Mount **SyncInitializer** (initState) ↓ Check **isFirstInstallAndEmpty()** ■■■ **TRUE** (Fresh install / unauthenticated legacy boxes empty) ■ ↓ ■ Set **_isFirstInstallSyncing = true** ■ ↓ ■ Render Full-Screen **'Syncing your data...'** ■ ↓ ■ Execute **performFullSync()** (Firestore read) ■ ↓ ■ **Exception / Offline Error → rethrow** ■ ↓ ■ **setState(false) SKIPPED → PERMANENTLY STUCK** ■■■ **FALSE** → Render ProjectArcApp / Login

3. Systematic Trace Matrix (Questions 1–17)

#	Question / Stage	Audited Behavior & Technical Mechanism	Code Location Reference
1-3	App Start & Checks	Initializes Flutter, Firebase Core, Hive, and Notifications. Mounts ProviderScope → SyncInitializer.	`lib/main.dart` → `main()`
4-6	Firebase Auth & UID	UserContextNotifier checks FirebaseAuth.instance.currentUser synchronously and listens to authStateChanges(). Sets UID.	`user_context_provider.dart` → `UserContextNotifier`
7-8	Hive Box Sizing	HiveService opens user-scoped boxes (e.g. tasks_box_uid_<UID>). Local data is loaded directly from disk.	`hive_service.dart` → `ensureUserBoxesOpened()`
9-10	Sync Trigger	SyncInitializer calls isFirstInstallAndEmpty(). If true, sets _isFirstInstallSyncing=true and renders 'Syncing your data...'.	`lib/main.dart` → `_startSyncProcess()`
11-13	Sync Finish & Next Screen	Requires pullInitialDataFromFirestore() to succeed cleanly. On success, setState(false) mounts ProjectArcApp (/login or /).	`lib/main.dart` → `build()`
14-15	Sync Failure / Offline	Firestore timeout/error is rethrown by pullInitialDataFromFirestore and performFullSync. Bypasses setState(false) → STUCK.	`sync_service.dart (L195)` `sync_manager.dart (L87)`
16-17	Unauthenticated User	UserContext is legacy (userId=null). isFirstInstallAndEmpty checks empty legacy boxes → returns true → attempts sync before login!	`sync_service.dart (L65)` `user_context.dart (L15)`

Project Arc — Login & Synchronization Audit

Detailed Failure Analysis, Component Audit & Technical Verdict

1. Component Deep-Dive Audit

Component	Implementation Details & State Handling	File / Code Location
A. Login Flow	LoginScreen maps username to synthetic email (<username>@projectarc.app) and invokes AuthService.loginWithUsername via signInWithEmailAndPassword. On success, calls GoRouter.go('/').	`login_screen.dart (L12)` `auth_service.dart (L58)`
B. Session Restoration	FirebaseAuth restores session on startup. UserContextNotifier receives user object, opens user-scoped boxes (<box>_uid_<uid>), and updates UserContext state.	`user_context_provider.dart (L18)` `hive_service.dart (L64)`
C. Sync Engine	SyncManager enforces 12h cooldown and 24h rolling limit (max 2 syncs). SyncService executes pullInitialDataFromFirestore across 7 collections (tasks, habits, entries, txs, budgets, summaries, settings).	`sync_manager.dart (L26)` `sync_service.dart (L91)`
D. 'Syncing your data...'	Displayed by SyncInitializer when _isFirstInstallSyncing==true. Has NO timeout, NO fallback UI, and NO error boundary. Keeps GoRouter unmounted.	`lib/main.dart (L120-150)` `SyncInitializer.build()`

2. Exact Mechanics of the Infinite Loading Freeze

Root Cause Trace:

1. On fresh launch, SyncInitializer._startSyncProcess() runs in initState().
2. syncService.isFirstInstallAndEmpty() evaluates to true because initial_sync_completed is false and legacy/null user Hive boxes are empty.
3. _isFirstInstallSyncing is set to true, replacing the application router with the full-screen *'Syncing your data...'* splash.
4. syncManager.performFullSync() invokes pullInitialDataFromFirestore().
5. If the device is offline, unauthenticated, or experiences network delay/Firestore exception, pullInitialDataFromFirestore() catches the error, logs a notice, and **RETHROWS**.
6. performFullSync() catches the rethrown exception and **RETHROWS IT AGAIN**.
7. _startSyncProcess() has **NO try-catch block**. The rethrown exception terminates method execution immediately.
8. setState(() { _isFirstInstallSyncing = false; }) is **SKIPPED**.
9. _isFirstInstallSyncing remains true forever, locking the user out of the Login Screen and Main App.

3. Problem Evaluation Matrix (Questions E.1 – E.7)

#	Evaluation Question	Audit Finding & Verdict
E.1	Is current behavior expected?	**NO.** The app should never lock the UI on an unhandled background sync error.
E.2	Is the application actually stuck?	**YES.** The state variable _isFirstInstallSyncing remains true indefinitely.
E.3	Which component is responsible?	SyncInitializer in lib/main.dart (lacks try-catch & unauth check).
E.4	What condition prevents progression?	Unhandled exception in performFullSync() bypasses setState(false).
E.5	Is Login screen reachable?	**NO.** ProjectArcApp and GoRouter are not mounted while syncing.
E.6	Authentication vs. Sync Timing	User MUST authenticate FIRST, then pull user-scoped Firestore data.
E.7	Consistency with 5-Step Architecture	**INCONSISTENT.** Triggers unauthenticated Firestore reads before user login.

FINAL AUDIT VERDICT

■ STARTUP/LOGIN/SYNC FLOW HAS A BLOCKING ISSUE

Technical Summary: The application contains a critical blocking bug in lib/main.dart (SyncInitializer._startSyncProcess()). When a fresh install or unauthenticated session encounters an offline state, network delay, or Firestore permission exception during startup sync, the exception is rethrown without error handling, skipping setState(() { _isFirstInstallSyncing = false; }). This permanently locks the UI on the *'Syncing your data...'* splash, completely bypassing GoRouter and rendering the Login Screen unreachable.